

LE STRINGHE



DICHIARAZIONE

Una stringa è un vettore di caratteri il cui ultimo elemento è il carattere '\0'.

Il vettore di caratteri che rappresenta la stringa è quindi formato da un numero di elementi pari al numero di caratteri della stringa più uno (il carattere di fine stringa).

Per dichiarare una stringa contenente N caratteri occorre dichiarare un vettore di N+1 caratteri.

Ad esempio per dichiarare una variabile stringa che possa contenere il codice fiscale bisogna dichiarare un vettore di 17 caratteri (16 caratteri per memorizzare le cifre del codice fiscale più il carattere di fine stringa ('\0'))

`char codice_fiscale[17];`



GETS E PUTS

gets(<variabile stringa>)

La funzione gets() acquisisce una stringa da tastiera compresi eventuali spazi e il ritorno a capo che trasforma nel carattere terminatore (\0).

puts(<stringa>)

Stampa <stringa> su schermo. Aggiunge sempre \n in coda alla stringa.

puts(s) è uguale a printf (%s\n,s);



Printf e scanf

Leggere e stampare una stringa

```
main(){
```

```
char s[21];
```

// stringa di max 20 caratteri + \0

```
printf("Inserisci una stringa: ");
```

```
scanf("%s", s);
```

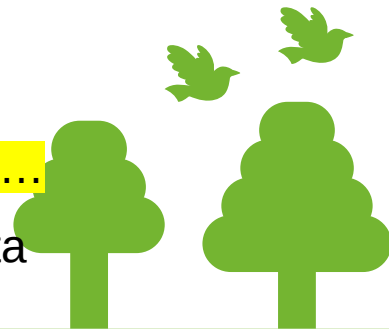
Identificatore di formato
%s

```
printf("Hai inserito la stringa: %s", s);
```

```
}
```

ATTENZIONE: `scanf("%s", s)` e **NON** `scanf("%s", &s)` ma non legge gli spazi...

Si può anche utilizzare `getchar` (libreria `stdio.h`) e leggere un carattere alla volta



Librerie

Per utilizzare le funzioni relative alla gestione delle stringhe è necessario includere nel proprio file sorgente l'header file `string.h` .

```
#include <string.h>
```



Funzione strlen

La funzione strlen(stringa) restituisce la lunghezza della stringa passata come parametro **escluso** il carattere di terminazione.

```
#include <string.h>
```

```
strlen(stringa);
```

Esempio:

```
int count
```

```
count= strlen("Hello")    count conterrà 5
```

Esercizio:

Scrivere un programma che chieda all'utente di inserire una parola e ne restituisca la lunghezza prima usando la funzione strlen e poi senza usare la funzione strlen.



Funzione strcpy

Le stringhe sono vettori di caratteri perciò non è possibile assegnare ad una variabile stringa un'altra variabile stringa utilizzando l'operatore =.

Si può quindi:

- copiare elemento per elemento dalla stringa sorgente alla stringa destinazione fino a quando si incontra il carattere di fine stringa con un ciclo FOR;
- utilizzare la funzione **strcpy**.

strcpy(destinazione,sorgente)

ATTENZIONE: la stringa destinazione deve essere abbastanza grande da contenere tutti gli elementi della stringa sorgente.

Esercizio

Scrivere un programma che letta una stringa immessa da tastiera la copia in un'altra stringa prima usando la funzione **strcpy** e poi senza utilizzare la funzione **strcpy**.



strcat

La funzione `strcat` consente di concatenare le due stringhe passate come parametro. Il risultato della concatenazione è memorizzato nella stringa passata come primo parametro.

`strcat(stringa1, stringa2)`

Se per esempio `stringa1` contiene "Hello" e `stringa2` contiene "World", allora dopo l'istruzione `strcat(stringa1, stringa2)` **`stringa1`** conterrà la stringa "HelloWorld".

Esercizio

Scrivere un programma che letti in ingresso nome e cognome di un utente li concateni in un'unica stringa prima usando la funzione **`strcat`** e poi senza utilizzare la funzione **`strcat`**.



strtok

La funzione **strtok** serve per spezzare una stringa *s* in base a dei caratteri di separazione. Spezza la stringa data in token divisi dal delimitatore specificato.

strtok accetta due argomenti:

- una stringa
- la stringa delimitatore

La funzione restituisce il puntatore a una stringa che rappresenta il token successivo.

Quando la stessa stringa viene analizzata con più chiamate strtok dopo la prima chiamata, il primo parametro deve essere NULL.



Esempio strtok

```
int main()
{
    char str[80] = "Ciao!Ciao!Bene !";
    char s[2] = "!";
    char *token;

    token = strtok(str, s);
    while( token != NULL )
    {
        printf( " %s\n", token );
        token = strtok(NULL, s);
    }
    return(0);
}
```

```
Ciao
Ciao
Bene
```

```
-----
Process exited after 0.09283 seconds with return value 0
Premere un tasto per continuare . . .
```



Esercizi

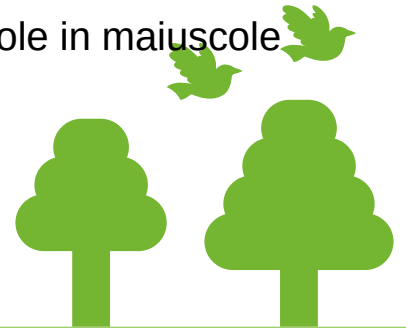
- Scrivere un programma che legga una stringa di massimo 50 caratteri contenete una frase separata da '-' e la scriva andando a capo ad ogni '-'. Es.: "Ciao-vado a capo-con un meno" diventa:

"Ciao"

"vado a capo"

"Con un meno"

- Scrivere un programma che preso in input una stringa trasformi le lettere minuscole in maiuscole



strcmp

La funzione strcmp confronta le due stringhe passate come argomento e restituisce un valore:

- = 0 se le due stringhe sono uguali
- <0 se la prima stringa precede alfabeticamente la seconda
- >0 se la seconda stringa precede alfabeticamente la prima.

```
#include <stdio.h>
#include <string.h>
```

```
if (strcmp(stringa1, stringa2) == 0)
    printf("le due stringhe sono uguali");
else if (strcmp(stringa1, stringa2) < 0)
    printf("%s < %s", stringa1, stringa2);
else printf("%s > %s", stringa1, stringa2);
}
```

Esercizio:

Scrivere un programma che chieda all'utente di inserire due stringhe e che stampi il risultato del confronto prima usando la funzione **strcmp** poi senza utilizzare la funzione **strcmp**.



tolower, toupper

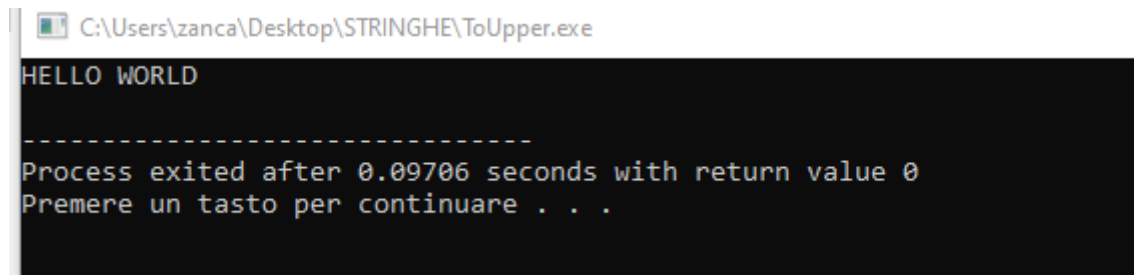
- **tolower()** converte una lettera maiuscola nella corrispondente lettera minuscola.
- **toupper()** converte una lettera minuscola nella corrispondente lettera maiuscola.

Se il carattere non ha un corrispondente carattere maiuscolo/minuscolo viene restituito il carattere invariato.

Bisogna includere ctype.h **#include ctype.h**

(con Dev funziona anche senza)

```
int main(){
    int j = 0;
    char str[50] = "Hello World";
    char str1[50];
    char ch;
    while (str[j]!='\0') {
        ch=toupper(str[j]);
        str1[j] =ch;
        j++;
    }
    puts(str1);
    return 0;
}
```



C:\Users\zanca\Desktop\STRINGHE\ToUpper.exe

HELLO WORLD

Process exited after 0.09706 seconds with return value 0
Premere un tasto per continuare . . .



atoi, atof

Atoi  converte una stringa in un intero

Atof  converte una stringa in un numero in virgola mobile

la stringa deve contenere un numero intero valido in caso contrario il risultato è indefinito. Gli spazi iniziali vengono ignorati e il numero può essere concluso da un qualsiasi carattere che non è valido come numero (spazio, lettera, punteggiatura . . . anche più di uno).

atoi("123") restituisce l'intero 123

atoi("123.45") restituisce l'intero 123 (".45" viene ignorato)

atof("123.45") restituisce il float 123.45

E' necessario includere `stdlib.h`:

`#include <stdlib.h>`

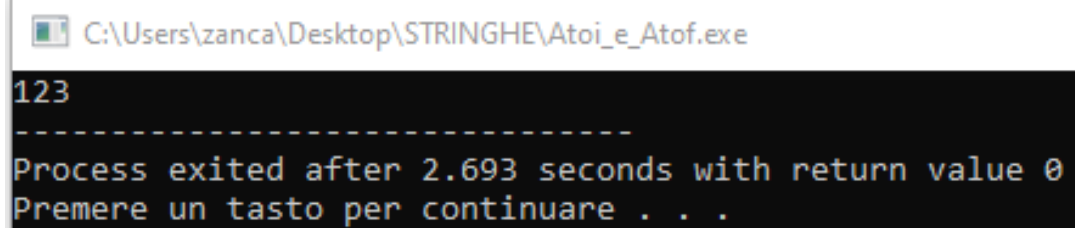


Esempio atoi

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>

int main(){
    int j = 0;
    char str[50] = " 123.4a!$";
    int num;
    num = atof(str);
    printf("%d",num);

    return 0;
}
```



C:\Users\zanca\Desktop\STRINGHE\Atoi_e_Atof.exe

123

Process exited after 2.693 seconds with return value 0

Premere un tasto per continuare . . .



Elenco funzioni

- `strlen(stringa)` **restituisce la lunghezza di una stringa**
- `strcpy(destinazione, sorgente)` **copia la stringa sorgente nella stringa destinazione**
- `strcat (stringa1,stringa2)` **concatena due stringhe (risultato nella prima)**
- `strtok(stringa,delimitatore)` **restituisce un puntatore alla stringa senza il delimit.**
- `strcmp` **restituisce =0, >0, <0 in base al confronto**
- `tolower/toupper` **converte un carattere(lettera) in minuscolo/maiuscolo**
- `atoi/atof` **converte una stringa di caratteri in intero/float**

